

Reviewer Pack — Minimal Order of Affine Generators and Communication Complex...

Entry cc5083a03737 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 22:23:17 UTC

Minimal Order of Affine Generators and Communication Complexity

Entry ID: cc5083a03737 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-31 22:23:17 UTC

1. Conjecture

Field A (mathematical branch): Affine Geometry **Field B** (complexity object): Communication Complexity

Statement:

The minimal number of affine generators required to represent a given set of boolean functions in an affine space is linearly correlated with the communication complexity of these functions, such that the minimum number of affine generators $m(G) = \Theta(\text{communication_complexity}(G))$ for any boolean function graph G .

Rationale (proposer's reasoning):

Affine geometry provides a way to encode boolean functions through their representations as points and hyperplanes in an affine space. By studying the minimal set of generators needed to represent these functions, we might uncover structural properties that relate to the complexity of communication tasks involving these functions.

Taxonomy category: AffineGeometry-CommunicationComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f3bb59e031bef499

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient (r) between the minimal number of affine generators $m(G)$ and communication complexity of boolean function graph G exceeds 0.7 for at least 80% of the seeds, with $p\text{-value} < 0.05$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function_graph(n):
        G = []
        for i in range(2**n):
            G.append([i ^ j for j in range(i+1)])
        return G

    def min_affine_generators(G):
        m = len(G)
        n = int(math.log2(m))
        A = [[0]*n for _ in range(m)]
        b = [0]*m

        for i, g in enumerate(G):
            for j in range(n):
                if (1 << j) & i:
                    A[i][j] = 1
                else:
                    A[i][j] = -1
            b[i] = 1

    # Gaussian elimination to find the rank of matrix A
    def gaussian_elimination(A, b):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            b[i], b[max_row] = b[max_row], b[i]
```

```

        for j in range(i+1, m):
            factor = -A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] += factor * A[i][k]
                b[j] += factor * b[i]

    rank = 0
    for i in range(m):
        if any(A[i][j] != 0 for j in range(n)):
            rank += 1
    return rank

rank = gaussian_elimination(A, b)
return rank

def communication_complexity(G):
    n = int(math.log2(len(G)))
    return n + 1

results = []
for n in [5, 10, 15, 20, 30, 40]:
    G = generate_boolean_function_graph(n)
    m_G = min_affine_generators(G)
    comm_complexity = communication_complexity(G)

    if m_G == 0 or comm_complexity == 0:
        continue

    results.append((m_G, comm_complexity))

if not results:
    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": None,
        "instances_tested": 1,
        "n_max": 40,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

m_G_values = [m for m, _ in results]
comm_complexity_values = [c for _, c in results]

mean_ranks = sum(m_G_values) / len(m_G_values)
std_dev = math.sqrt(sum((x - mean_ranks)**2 for x in m_G_values) / len(m_G_values))
correlation_coefficient = sum((m_G_values[i] - mean_ranks) * (comm_complexity_values[i] - sum(comm_complexity_values) / len(comm_complexity_values))) / (len(m_G_values) - 1)

return {
    "metric_name": "Pearson correlation coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(results),
    "n_max": 40,
    "conjecture_holds": abs(correlation_coefficient) >= 0.7,
    "counterexample": ""
}

```

```

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_ranks = [r["metric_value"] for r in results if r["metric_value"] is not None]
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={sum(mean_ranks)/len(mean_ranks)} std={math.sqrt(sum((x - s)**2 for x, s in zip(mean_ranks, mean_ranks)))/len(mean_ranks)}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={sum(mean_ranks)/len(mean_ranks)} std={math.sqrt(sum((x - s)**2 for x, s in zip(mean_ranks, mean_ranks)))/len(mean_ranks)}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_aec7aa5a.py", line 114, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_aec7aa5a.py", line 74, in run_trial
    m_G = min_affine_generators(G)
          ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_aec7aa5a.py", line 64, in min_affine_generators
    rank = gaussian_elimination(A, b)
           ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_aec7aa5a.py", line 47, in gaussian_elimination
    if abs(A[j][i]) > abs(A[max_row][i]):
        ~~~~^~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it was unable to compute the Pearson correlation coefficient and p-value as required by the support condition. | next: Re-run the test with proper error handling to ensure that it can complete without crashing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13884
2	propose	ollama_remote	glm4:latest	0	0	24976
3	preregistration	ollama_remote	glm4:latest	0	0	16328
4	novelty	ollama_remote	glm4:latest	0	0	8306
5	novelty	ollama_remote	glm4:latest	0	0	12907
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14354
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	102923
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	42449
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13977
10	judge	ollama_remote	glm4:latest	0	0	14494

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 264598 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/cc5083a03737.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cc5083a03737.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/cc5083a03737.tar.gz` (if generated)